Programação Orientada a Objetos

Parte 1

Classes, Objetos, Encapsulamento

Prof. Geovani

03/03/2026

Marcos Importantes

Data	Evento	Detalhes
03/02	■ Aula 1	Apresentação (Realizada)
17/03	■ Entrega 1	Proposta do Projeto
05/05	■ AP (30%)	Avaliação Parcial
26/05	■ Entrega 2	Modelagem do BD
02/06	■ Aula 16	JWT - ÚLTIMA AULA
09/06	■ AF (30%)	APRESENTAÇÃO FINAL
16/06	■ AI (40%)	Avaliação Interdisciplinar
23/06	■ AIS	Substitutiva (se aplicável)
30/06	■ Aula 20	Encerramento

Atenção ao nosso cronograma

Aula	Data	Conteúdo Principal	Evento/Entrega
1	03/02	Apresentação da Disciplina	
2	10/02	Fundamentos C# - Parte 1	-
-	17/02	CARNAVAL - Sem aula	Feriado
~~3~~	~~24/02~~	~~Fundamentos C# Parte 2 + Coleções~~	
4	03/03	OOP - Parte 1	-
5	10/03	OOP - Parte 2	-
6	17/03	.NET Core e ASP.NET Core	Entrega Proposta de Projeto
7	24/03	SOLID + Design Patterns	-
8	31/03	Clean Architecture	-
-	03-04/04	Paixão de Cristo - Suspensão	Feriado
9	07/04	HTTP e REST	-
10	14/04	Construindo APIs REST	-
-	21/04	Tiradentes - Feriado	Feriado
11	28/04	Middleware e Filtros	-
12	05/05	AVALIAÇÃO PARCIAL (AP)	AP - 30%
13	12/05	PostgreSQL + EF Core	-
14	19/05	Repository Pattern + BD	-
15	26/05	Queries Avançadas	Entrega da Arquitetura de Software do projeto
16	**02/06**	**AUTENTICAÇÃO JWT**	**FIM CONTEÚDO**
-	04-06/06	Corpus Christi - Suspensão	Feriado
17	09/06	APRESENTAÇÃO FINAL (AF)	AF - 30%
18	16/06	AVALIAÇÃO INTERDISCIPLINAR (AI)	AI - 40%
19	23/06	Avaliação Substitutiva (AIS)	AIS
20	30/06	Encerramento + Revisão	Fim Semestre

Objetivos de Hoje

- Entender o Paradigma OOP
- Criar Classes e Objetos em C#
- Dominar Fields vs Properties
- Implementar Encapsulamento correto
- Trabalhar com Construtores
- Aplicar Modificadores de Acesso
- Usar this keyword e métodos estáticos

Revisão: Aula 3 - Fundamentos C# Parte 2

- ✅ Arrays e Arrays Multidimensionais
- ✅ Coleções Genéricas (List, Dictionary, HashSet)
- ✅ Manipulação de Strings e StringBuilder
- ✅ Métodos e Funções
- ✅ Tratamento de Exceções (try-catch-finally)
- ✅ Introdução ao LINQ (Where, Select)

O que é OOP?

Programação Orientada a Objetos

Paradigma OOP - Conceitos Fundamentais

Paradigma de Programação

Nova forma de pensar e estruturar código

Objetivos principais:

- Modelar o mundo real em código
- Organizar sistemas complexos
- Reutilizar código de forma eficiente
- Facilitar manutenção e evolução

Tudo são OBJETOS que interagem entre si!

Objetos têm características (propriedades) e comportamentos (métodos)

Os 4 Pilares da OOP

1 Abstração
Simplificar a realidade
Focar no essencial

2 Encapsulamento
Proteger dados
Ocultar complexidade

3 Herança
Reutilizar código
Relação "é um(a)"

4 Polimorfismo
Múltiplas formas
Mesma interface

Abstração - Simplificar a Realidade

Conceito:

Focar apenas no que é ESSENCIAL

Ignorar detalhes desnecessários para o contexto

Exemplo: Classe Carro

- Essencial: marca, modelo, ano, cor, placa
- Ignorar: peso dos parafusos, tipo de tinta específica

Na prática:

Definir apenas os atributos e métodos relevantes

para o CONTEXTO do sistema

Classes e Objetos em C#

📘 CLASSE = Molde/Template

- Define estrutura e comportamento
- Modelo abstrato
- Não ocupa memória até instanciar
- Palavra-chave: class

Exemplo: Classe Pessoa

🎯 OBJETO = Instância

- Criado a partir da classe
- Tem valores concretos
- Ocupa memória (heap)
- Palavra-chave: new

Exemplo: João, Maria, Pedro

💡 **Analogia: Classe = Forma de bolo | Objeto = Bolo pronto**

Uma CLASSE pode criar INFINITOS objetos!

Anatomia de uma Classe em C#

```
public class Pessoa
{
    // 1. CAMPOS (Fields) - dados privados internos
    private string nome;
    private int idade;

    // 2. PROPRIEDADES (Properties) - interface pública
    public string Nome { get; set; }
    public int Idade { get; set; }

    // 3. CONSTRUTORES - inicialização de objetos
    public Pessoa(string nome, int idade)
    {
        Nome = nome;
        Idade = idade;
    }

    // 4. MÉTODOS - comportamentos/ações
    public void Apresentar()
    {
        Console.WriteLine($"Olá, sou {Nome}, tenho {Idade} anos");
    }
}
```

Criando Objetos (Instâncias)

```
// Criando objetos usando NEW
Pessoa pessoa1 = new Pessoa("João", 25);
Pessoa pessoa2 = new Pessoa("Maria", 30);
Pessoa pessoa3 = new Pessoa("Pedro", 28);

// Acessando propriedades
Console.WriteLine(pessoa1.Nome);  // João
Console.WriteLine(pessoa2.Idade); // 30

// Modificando propriedades
pessoa1.Idade = 26;
pessoa2.Nome = "Maria Silva";

// Chamando métodos
pessoa1.Apresentar();  // Olá, sou João, tenho 26 anos
pessoa2.Apresentar();  // Olá, sou Maria Silva, tenho 30 anos
pessoa3.Apresentar();  // Olá, sou Pedro, tenho 28 anos

// ⚠️ IMPORTANTE: Cada objeto tem SUA PRÓPRIA memória!
```

Fields vs Properties

Campos e Propriedades

Fields (Campos) - Dados Internos

```
public class Produto
{
    // Fields (campos) - geralmente PRIVATE
    private string nome;
    private decimal preco;
    private int estoque;

    // ✅ Características:
    // • Armazenam dados da classe
    // • Geralmente private (encapsulados)
    // • Nomenclatura: camelCase
    // • Acesso direto (sem get/set)
    // • Mais rápidos (menos overhead)
    // • Sem validação automática
}

// ❌ NÃO RECOMENDADO: campos públicos
public class ProdutoRuim
{
    public string nome;   // Sem proteção!
    public decimal preco; // Qualquer um pode alterar!
    // Viola o encapsulamento!
}
```

Properties (Propriedades) - Interface Pública

```
public class Produto
{
    private decimal preco;  // Campo privado

    // Property - acesso controlado ao campo
    public decimal Preco
    {
        get { return preco; }  // Leitura (getter)
        set                     // Escrita (setter)
        {
            if (value < 0)
                throw new ArgumentException("Preço não pode ser negativo!");
            preco = value;
        }
    }
}

// ✅ Características:
// • Interface pública para acessar dados
// • Nomenclatura: PascalCase
// • Podem ter validação (get/set)
// • Encapsulam campos privados
// • RECOMENDADO para expor dados
```

Fields vs Properties - Comparação

Característica	Fields (Campos)	Properties (Propriedades)
Visibilidade	Geralmente private	Geralmente public
Sintaxe	private int idade;	public int Idade { get; set; }
Validação	Não tem	Pode ter (no setter)
Nomenclatura	camelCase	PascalCase
Uso	Dados internos	Interface pública
Performance	Mais rápido	Ligeiramente mais lento
Recomendação	Dados privados	✅ Expor dados externamente

💡 **BOA PRÁTICA: Use Properties para expor dados!**

Auto-Properties (Propriedades Automáticas)

```
// FORMA ANTIGA (verbosa)
private string nome;

public string Nome
{
    get { return nome; }
    set { nome = value; }
}

// Muito código repetitivo! 😓
```

```
// FORMA MODERNA ✅
public string Nome { get; set; }

// UMA LINHA APENAS! 🎉
// Compilador cria campo
// automaticamente nos bastidores

// Mais exemplos:
public int Idade { get; set; }
public string Email { get; set; }
public decimal Salario { get; set; }
public DateTime DataNasc { get; set; }

// Limpo, conciso e moderno!
```

Property com Validação - Exemplo Prático

```
public class Produto
{
    private decimal preco;
    private int estoque;

    public decimal Preco
    {
        get { return preco; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Preço não pode ser negativo!");
            if (value > 1000000)
                throw new ArgumentException("Preço muito alto!");
            preco = value;
        }
    }

    public int Estoque
    {
        get { return estoque; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Estoque não pode ser negativo!");
            estoque = value;
        }
    }
}
```

Read-only Properties (get; init;)

```
public class Pessoa
{
    // Somente leitura - set apenas no construtor
    public string CPF { get; init; }

    // Read-only tradicional (sem init)
    public DateTime DataCriacao { get; }

    // Propriedade normal
    public string Nome { get; set; }

    public Pessoa(string cpf, string nome)
    {
        CPF = cpf;  // Só pode setar aqui
        Nome = nome;
        DataCriacao = DateTime.Now;
    }
}

// Uso:
Pessoa p = new Pessoa("123.456.789-00", "João");
Console.WriteLine(p.CPF);  // OK: ler
p.Nome = "João Silva";      // OK: modificar
p.CPF = "000.000.000-00";   //  ❌ ERRO! CPF é init-only
p.DataCriacao = DateTime.Now; //  ❌ ERRO! Sem setter
```

Construtores

Inicializando Objetos

Construtores - Conceito

O que são Construtores?

Métodos especiais para INICIALIZAR objetos

Chamados automaticamente ao criar instância (new)

Características:

- Mesmo nome da classe
- Não tem tipo de retorno (nem void)
- Executados automaticamente ao usar new
- Podem receber parâmetros
- Classe pode ter vários construtores

Objetivo:

Garantir que objeto seja criado em estado VÁLIDO

Tipos de Construtores

```
public class Produto
{
    public string Nome { get; set; }
    public decimal Preco { get; set; }

    // 1. Construtor PADRÃO (sem parâmetros)
    public Produto()
    {
        Nome = "Sem nome";
        Preco = 0;
    }

    // 2. Construtor PARAMETRIZADO
    public Produto(string nome, decimal preco)
    {
        Nome = nome;
        Preco = preco;
    }

    // 3. Construtor com VALIDAÇÃO
    public Produto(string nome, decimal preco, bool validar)
    {
        if (validar && preco < 0)
            throw new ArgumentException("Preço inválido!");
        Nome = nome;
        Preco = preco;
    }
}
```

Sobrecarga de Construtores

```
public class Cliente
{
    public string Nome { get; set; }
    public string Email { get; set; }
    public string CPF { get; set; }

    // Construtor 1: Só nome
    public Cliente(string nome)
    {
        Nome = nome;
    }

    // Construtor 2: Nome e email
    public Cliente(string nome, string email)
    {
        Nome = nome;
        Email = email;
    }

    // Construtor 3: Completo
    public Cliente(string nome, string email, string cpf)
    {
        Nome = nome;
        Email = email;
        CPF = cpf;
    }
}

// Uso:
Cliente c1 = new Cliente("João");
Cliente c2 = new Cliente("Maria", "maria@email.com");
Cliente c3 = new Cliente("Pedro", "pedro@email.com", "123.456.789-00");
```

Encapsulamento

Protegendo Dados

Encapsulamento - Conceito Fundamental

Definição:

Ocultar detalhes internos da implementação

Expor apenas interface pública necessária

Objetivos:

- Proteger dados contra acesso indevido
- Ocultar complexidade interna
- Facilitar manutenção
- Aumentar segurança do código

Como fazer:

- Campos privados (private)
- Properties públicas (public)
- Validações nos setters
- Métodos privados para lógica interna

Por que Encapsular?

❌ SEM Encapsulamento:

```
public class ContaBancaria
{
    public decimal saldo;  // Perigo!
}

var conta = new ContaBancaria();
conta.saldo = -1000;  // 😱 PROBLEMA!
// Qualquer um pode colocar
// valor negativo!
// Sem controle, sem validação!
```

✅ COM Encapsulamento:

```
public class ContaBancaria
{
    private decimal saldo;

    public decimal Saldo
    {
        get { return saldo; }
        private set { saldo = value; }
    }

    public void Depositar(decimal v)
    {
        if (v > 0) saldo += v;
    }
}
// Acesso controlado! 🎉
```

Modificadores de Acesso em C#

Modificador	Descrição	Acesso
public	Acesso total	Qualquer lugar
private	Acesso restrito	Apenas dentro da classe
protected	Acesso herança	Classe e classes derivadas
internal	Acesso assembly	Dentro do mesmo projeto
protected internal	União dos dois	Projeto OU herança
private protected	Interseção	Projeto E herança

💡 **Regra de Ouro: Use o mais RESTRITIVO possível!**
Default: private para campos, public para métodos/properties

Métodos

Comportamentos e Ações

Métodos - Comportamentos da Classe

```
public class Calculadora
{
    // Método que retorna valor
    public int Somar(int a, int b)
    {
        return a + b;
    }

    // Método void (não retorna)
    public void Imprimir(string mensagem)
    {
        Console.WriteLine(mensagem);
    }

    // Método com múltiplos parâmetros
    public double CalcularMedia(double nota1, double nota2, double nota3)
    {
        return (nota1 + nota2 + nota3) / 3;
    }

    // Método privado (auxiliar)
    private bool ValidarNumero(int numero)
    {
        return numero > 0;
    }
}
```

Métodos de Instância vs Métodos Estáticos

```
// MÉTODOS DE INSTÂNCIA
public class Pessoa
{
    public string Nome { get; set; }

    // Precisa de objeto
    public void Apresentar()
    {
        Console.WriteLine(
          $"Olá, {Nome}");
    }
}

// Uso:
Pessoa p = new Pessoa();
p.Nome = "João";
p.Apresentar();  // OK
```

```
// MÉTODOS ESTÁTICOS
public class Matematica
{
    // static - sem objeto
    public static int Somar(
      int a, int b)
    {
        return a + b;
    }
}

// Uso: direto pela classe
int resultado =
  Matematica.Somar(5, 3);

// Não precisa new! 🎉
```

this Keyword - Referência ao Objeto Atual

```
public class Produto
{
    private string nome;
    private decimal preco;

    // this = referência ao objeto atual
    public Produto(string nome, decimal preco)
    {
        this.nome = nome;   // this.nome = campo, nome = parâmetro
        this.preco = preco; // Evita conflito de nomes!
    }

    // this para chamar outro construtor
    public Produto(string nome) : this(nome, 0)
    {
        // Chama o construtor acima
    }

    // this para retornar o próprio objeto (fluent interface)
    public Produto DefinirNome(string nome)
    {
        this.nome = nome;
        return this;  // Permite encadeamento
    }
}

// Uso do encadeamento:
var p = new Produto().DefinirNome("Notebook").DefinirPreco(3000);
```

Sobrecarga de Métodos (Overload)

```
public class Calculadora
{
    // Mesmo nome, assinaturas diferentes!

    // Versão 1: dois inteiros
    public int Somar(int a, int b)
    {
        return a + b;
    }

    // Versão 2: três inteiros
    public int Somar(int a, int b, int c)
    {
        return a + b + c;
    }

    // Versão 3: doubles
    public double Somar(double a, double b)
    {
        return a + b;
    }

    // Versão 4: array
    public int Somar(params int[] numeros)
    {
        return numeros.Sum();
    }
}

// Uso: compilador escolhe automaticamente
var calc = new Calculadora();
calc.Somar(1, 2);        // Chama versão 1
calc.Somar(1, 2, 3);     // Chama versão 2
calc.Somar(1.5, 2.5);    // Chama versão 3
```

Membros Estáticos

static keyword

Palavra-chave static - Membros da Classe

O que é static?

Pertence à CLASSE, não ao objeto

Compartilhado por todas as instâncias

Acessado sem criar objeto (new)

Quando usar:

- Métodos utilitários (Math, Console)
- Contadores globais
- Constantes
- Factory methods

Cuidados:

- Não pode acessar membros de instância
- Não pode usar this
- Evite abusar (dificulta testes)

Membros Estáticos - Exemplo Completo

```
public class Contador
{
    // Campo estático - compartilhado
    private static int totalObjetos = 0;

    // Campo de instância - individual
    private int numero;

    // Propriedade estática
    public static int TotalObjetos
    {
        get { return totalObjetos; }
    }

    // Construtor
    public Contador()
    {
        totalObjetos++;  // Incrementa contador global
        numero = totalObjetos;
    }

    // Método estático
    public static void Zerar()
    {
        totalObjetos = 0;
    }
}

// Uso:
Console.WriteLine(Contador.TotalObjetos); // 0
var c1 = new Contador();
var c2 = new Contador();
Console.WriteLine(Contador.TotalObjetos); // 2
```

Classes Estáticas - Utility Classes

```
// Classe totalmente estática
public static class Utilidades
{
    // Todos os membros devem ser static

    public static string CapitalizarPrimeiraLetra(string texto)
    {
        if (string.IsNullOrEmpty(texto))
            return texto;
        return char.ToUpper(texto[0]) + texto.Substring(1).ToLower();
    }

    public static bool ValidarCPF(string cpf)
    {
        // lógica de validação
        return cpf.Length == 11;
    }

    public static decimal CalcularDesconto(decimal valor, decimal percentual)
    {
        return valor * (1 - percentual / 100);
    }
}

// Uso: sem new!
string nome = Utilidades.CapitalizarPrimeiraLetra("joão");
bool valido = Utilidades.ValidarCPF("12345678900");
decimal preco = Utilidades.CalcularDesconto(100, 10);
```

Hora de Praticar!

Exercícios Práticos

Exercício 1: Criar Classes de Domínio

DESAFIO:
Criar classes para sistema de e-commerce

Requisitos:

1. Classe Produto
   - Id, Nome, Preco, Estoque
   - Validação: Preço > 0, Estoque >= 0
   - Método: DiminuirEstoque(quantidade)

2. Classe Cliente
   - Id, Nome, Email, CPF
   - Validação: Email válido, CPF 11 dígitos
   - Método: Apresentar()

3. Classe Pedido
   - Id, Cliente, Lista<Produto>, Data, Total
   - Método: CalcularTotal()
   - Método: AdicionarProduto(produto)

Exercício 2: Implementar Encapsulamento

DESAFIO:
Refatorar classe para aplicar encapsulamento

❌ Código Ruim (SEM encapsulamento):

```
public class ContaBancaria
{
    public string titular;
    public decimal saldo;
}
```

✅ Refatore para:

- Campos privados
- Properties com validação
- Métodos Depositar e Sacar
- Validações adequadas
- Não permitir saldo negativo

Exercício 3: Relacionamentos entre Objetos

DESAFIO:

Implementar relacionamento entre classes

📋 Criar:

1️⃣ Classe Autor

- Nome, Email, Biografia

2️⃣ Classe Livro

- Titulo, Autor (objeto), Preco, Paginas
- Método: ExibirDetalhes()

3️⃣ No método ExibirDetalhes:

- Mostrar título do livro
- Mostrar nome do autor
- Mostrar preço e páginas

💡 Objetivo: Praticar composição de objetos!

Fim de Jogo

https://w.app/gil

Obrigado e até a próxima aula!